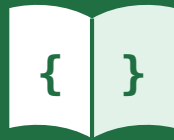

Modül 04 — HTTP İstemcileri ve Dayanıklılık

Ders Notları

Spring Boot Edu



2026-09-23

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Spring'de HTTP İstemcileri	2
2.2 Dağıtık Sistemlerde Hatalar	3
3. Adım Adım Örnekler	3
3.1 RestClient	3
3.2 HTTP Hatalarını Domain Exception'larına Çevirmek	4
3.3 Tüm İstemciler İçin Ortak Ayarlar	5
3.4 HTTP Interface İstemcileri	5
3.5 Karşılaştırma: WebClient	6
3.6 @Retryable ile Tekrar Deneme (Spring Framework 7)	7
3.7 @ConcurrencyLimit ile Aşırı Yüku Önlemek	8
3.8 Zaman Aşımaları	9
4. Sık Yapılan Hatalar ve En İyi Pratikler	10
5. Özet	10
6. İleri Okuma	11

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Başka bir servisi `RestClient` ile çağırmak ve HTTP hatalarını domain exception'larına çevirmek
- Tüm istemcilere ortak başlık ve interceptor eklemek
- Uygulama kodu yazmadan, yalnızca bir arayüzle HTTP istemcisi tanımlamak (HTTP interface)
- `WebClient` ile reaktif çağrının farkını görmek
- Spring Framework 7'nin `@Retryable` ve `@ConcurrencyLimit` açıklamalarıyla geçici hatalara ve aşırı yüke karşı dayanıklı olmak
- Zaman aşımalarını ayarlamak
- Dış servisleri `WireMock` ile testte taklit etmek

Ön koşullar: Modül 01-03 · **Tahmini süre:** 4 saat

2. Kavramlar

2.1 Spring'de HTTP İstemcileri

İstemci	Model	Ne zaman?
<code>RestClient</code>	Senkron, akıcı API	Varsayılan tercih. Virtual thread'lerle birlikte ölçeklenir
HTTP interface (<code>@HttpExchange</code>)	Arayüz, uygulamayı Spring üretir	Çok sayıda uç noktası olan bir servisi çağırırken
<code>WebClient</code>	Reaktif (Mono/Flux)	Uygulama zaten reaktifse (Modül 13)
<code>RestTemplate</code>	Eski, senkron	Yeni kodda kullanmayın

Spring Boot, `RestClient.Builder` bean'ini hazır verir: JSON dönüştürücüleri, zaman aşım-ları ve `RestClientCustomizer`'lar zaten uygulanmıştır. Alttaki HTTP kütüphanesini de sınıf yolundan seçer. Tercih sırası Apache HttpClient, Jetty, Reactor Netty, JDK HttpClient, JDK HttpURLConnection şeklindedir. `spring.http.clients.imperative.factory` ile bu seçim sa-bitlenebilir.

2.2 Dağıtık Sistemlerde Hatalar

Uzak bir servis yavaşlayabilir, geçici olarak 503 dönebilir ya da hiç cevap vermeyebilir. Dayanıklılık (resilience) desenleri bu durumlara hazırlıklı olmayı sağlar:

Desen	Ne yapar?	Bu modülde
Timeout	Sonsuza kadar beklemeyi önler	<code>spring.http.clients.read-timeout</code>
Retry	Geçici hatalarda bekleyip tekrar dener	<code>@Retryable</code>
Bulkhead / eşzamanlılık sınırı	Aynı anda yapılan çağrı sayısını sınırlar	<code>@ConcurrencyLimit</code>
Fallback	Başarısızlıkta anlamlı bir yedek cevap verir	<code>try/catch</code> + son bilinen değer

! Önemli

Yalnızca **geçici** hataları tekrar deneyin (5xx, zaman aşımı). 404 veya 400 tekrar denemekle düzelmez. Ayrıca tekrar denenen işlemin idempotent olduğundan emin olun.

3. Adım Adım Örnekler

Uygulamayı başlatın:

```
./mvnw -pl modules/04-http-clients-resilience/lesson -am spring-boot:run
```

“Uzak” katalog servisi, ders uygulamasının içinde `FakeCatalogController` olarak çalışır. Fiyat uç noktası her ISBN için iki kez 503 döner. Kapak uç noktası ise 200 ms sürer. İstemciler bu servise gerçek HTTP ile bağlanır. Testlerde bu servisin yerini WireMock alır.

3.1 RestClient

Amaç: Başka bir servisten JSON okuyup bir record’a çevirmek.

```
public CatalogRestClient(RestClient.Builder builder, CatalogProperties properties) {
    this.restClient = builder // Boot's builder: timeouts,
    customizers, JSON
    .baseUrl(properties.baseUrl())
    .build();
}
```

```
}

public BookInfo find(String isbn) {
    return restClient.get()
        .uri("/catalog/books/{isbn}", isbn)           // URI template: the value is
        ↪ encoded safely
        .retrieve()
        .onStatus(status -> status.value() == 404, (request, response) -> {
            throw new BookInfoNotFoundException(isbn);
        })
        .onStatus(HttpStatus::is5xxServerError, (request, response) -> {
            throw new CatalogUnavailableException("Catalog answered " +
                ↪ response.getStatusCode());
        })
        .body(BookInfo.class);                       // JSON → record
}
```

Beklenen çıktı:

```
== 3.1 RestClient
BookInfo[isbn=9780134685991, title=Effective Java, authors=[Joshua Bloch], pageCount=412]
```

Testi: catalog/CatalogRestClientTest. Uzak servisi WireMock taklit eder:

```
@Test
void readsABook() {
    cata-
    ↪ log.stubFor(get("/catalog/books/9780134685991").willReturn(okJson(EFFECTIVE_JAVA)));
    ↪ // fake response

    assertThat(client.find("9780134685991"))
        .isEqualTo(new BookInfo("9780134685991", "Effective Java",
            ↪ java.util.List.of("Joshua Bloch"), 412));
}
```

3.2 HTTP Hatalarını Domain Exception'larına Çevirmek

Amaç: Çağırın kod HTTP ayrıntılarını değil, anlamlı hataları görsün.

Bölüm 3.1'deki onStatus çağrıları bunu yapar: 404 → BookInfoNotFoundException, 5xx → CatalogUnavailableException. onStatus olmasaydı RestClient, HttpClientErrorException / HttpServerErrorException fırlatırdı.

Beklenen çıktı:

```
== 3.2 Error handling
```

```
BookInfoNotFoundException: The catalog has no book with ISBN 9780000000000
```

3.3 Tüm İstemciler İçin Ortak Ayarlar

Amaç: User-Agent ve istek kimliği gibi başlıkları tek bir yerde, tüm istemcilere eklemek.

Boot, her `RestClient.Builder`'a tüm `RestClientCustomizer` bean'lerini uygular:

```
@Component
public class CorrelationIdCustomizer implements RestClientCustomizer {

    @Override
    public void customize(RestClient.Builder builder) {
        builder.defaultHeader(HttpHeaders.USER_AGENT, "bookstore/1.0")
            .requestInterceptor((request, body, execution) -> {
                request.getHeaders().add("X-Request-Id",
↪      UUID.randomUUID().toString()); // trace a call across services
                return execution.execute(request, body);
            });
    }
}
```

Testi: `CatalogRestClientTest.everyRequestCarriesACorrelationIdAndUserAgent`. WireMock, gelen isteğin başlıklarını doğrular.

3.4 HTTP Interface İstemcileri

Amaç: Bir servisin uç noktalarını yalnızca bir arayüz ile tanımlamak. Uygulamayı Spring üretir.

```
@HttpExchange("/catalog/books")
public interface CatalogApi {

    @GetExchange("/{isbn}")
    BookInfo find(@PathVariable String isbn);

    @GetExchange("/{isbn}/price")
    Price price(@PathVariable String isbn);

    @GetExchange("/{isbn}/cover")
    byte[] cover(@PathVariable String isbn);
}
```

Spring Framework 7'nin `@ImportHttpServices`'i, arayüzleri bir **grup** olarak bean yapar:

```
@Configuration(proxyBeanMethods = false)
@ImportHttpServices(group = "catalog", types = CatalogApi.class)
@EnableConfigurationProperties(CatalogProperties.class)
public class HttpClientsConfiguration {
}
```

Grubun adresi ve zaman aşımaları `application.yaml` dosyasından gelir:

```
http:
  clients:                                # defaults for every HTTP client Boot creates (Lesson
  ↪ 3.8)
    connect-timeout: 2s
    read-timeout: 5s
    imperative:
      factory: jdk                        # RestClient on the JDK HttpClient (Reactor Netty is
  ↪ only for WebClient here)
    serviceclient:
      catalog:                           # the group of @ImportHttpServices(group = "catalog")
  ↪ (Lesson 3.4)
        base-url: http://localhost:${server.port:8080}
        read-timeout: 3s                  # overrides the default for this group only
```

Beklenen çıktı:

```
== 3.4 HTTP interface client
Spring in Action
```

Testi: `catalog/CatalogApiTest`

3.5 Karşılaştırma: WebClient

Amaç: Aynı çağrının reaktif sürümünü görmek.

```
@Component
public class ReactiveCatalogClient {

    private final WebClient webClient;

    public ReactiveCatalogClient(WebClient.Builder builder, CatalogProperties
  ↪ properties) {
        this.webClient = builder.baseUrl(properties.baseUrl()).build();
    }
}
```

```
public Mono<BookInfo> find(String isbn) {  
    return webClient.get()  
        .uri("/catalog/books/{isbn}", isbn)  
        .retrieve()  
        .bodyToMono(BookInfo.class);           // nothing happens until  
        ↪ someone subscribes  
}
```

Mono tembeldir: `block()` veya `subscribe()` çağrılana kadar istek gönderilmez. Servlet (Web MVC) uygulamasında virtual thread'lerle `RestClient` daha basit ve yeterlidir. `WebClient` reaktif uygulamalar içindir (Modül 13).

Beklenen çıktı:

```
== 3.5 WebClient  
Mono → 412 pages
```

i Not

`WebClient` starter'ı `Reactor Netty`'yi getirir. Boot, sınıf yolundaki bu kütüphaneyi `RestClient` için de seçerdi. Bu modülde `spring.http.clients.imperative.factory:jdk` ile `RestClient`, JDK'nın `HttpClient`'na sabitlenmiştir.

3.6 @Retryable ile Tekrar Deneme (Spring Framework 7)

Amaç: Geçici hatalarda artan beklemeyle tekrar denemek. Ek bir kütüphane gerekmez.

Önce özellik açılır:

```
@Configuration(proxyBeanMethods = false)  
@EnableResilientMethods  
public class ResilienceConfiguration {  
}
```

Tekrar denenen çağrı:

```
@Component  
public class PriceQuery {  
  
    private final CatalogApi catalog;  
  
    public PriceQuery(CatalogApi catalog) {  
        this.catalog = catalog;  
    }  
}
```

```

    }

    @Retryable(
        includes = HttpServerErrorException.class, // only 5xx: a 404 will not get
        ↪ better by retrying
        maxRetries = 3, // 1 call + up to 3 retries
        delay = 100, multiplier = 2, jitter = 20 // wait ~100 ms, ~200 ms, ~400 ms
        public Price fetch(String isbn) {
            return catalog.price(isbn);
        }
    }
}

```

Tüm denemeler başarısız olursa, çağırın taraf son bilinen fiyata döner:

```

public Price priceOrLastKnown(String isbn) {
    try {
        return currentPrice(isbn);
    } catch (RestClientException exception) {
        log.warn("Catalog unavailable after retries, using the last known price: {}",
        ↪ exception.getMessage());
        Price fallback = lastKnown.get(isbn);
        if (fallback == null) {
            throw exception;
        }
        return fallback;
    }
}

```

Beklenen çıktı (sahte katalog iki kez 503 döner):

```

== 3.6 @Retryable (the fake catalog fails twice, then answers)
price 89.90 after 335 ms

```

Testi: resilience/PriceServiceTest. WireMock’un scenario özelliğiyle “iki hata, sonra başarı” kurgulanır ve tam 3 istek geldiği doğrulanır. 404’ün tekrar denenmediği de test edilir.

! Uyarı

@Retryable bir proxy ile çalışır. PriceService içinden this.fetch(...) diye çağırırsaydık tekrar deneme **olmazdı** (self-invocation, Modül 01). Bu yüzden tekrar denenene metot ayrı bir bean’de, PriceQuery’dedir.

3.7 @ConcurrencyLimit ile Aşırı Yükü Önlemek

Amaç: Bir iş ortamının “aynı anda en fazla 2 istek” kuralına uymak.

```

@ConcurrencyLimit(2)                                // the 3rd caller waits (policy
↳ BLOCK, the default)
public byte[] download(String isbn) {
    maxInFlight.accumulateAndGet(inFlight.incrementAndGet(), Math::max);
    try {
        return catalog.cover(isbn);
    } finally {
        inFlight.decrementAndGet();
        completed.incrementAndGet();
    }
}

```

Varsayılan politika BLOCK'tur ve fazladan gelen çağırılar sıra bekler. `policy = REJECT` ise onları hemen `InvocationRejectedException` ile reddeder (Ödev 3). Virtual thread'lerle binlerce çağırılar olabileceği için bu sınır özellikle önemlidir.

Beklenen çıktı (her biri 200 ms süren 6 indirme):

```

== 3.7 @ConcurrencyLimit(2): 6 downloads of 200 ms each
max in flight 2, took 648 ms

```

Üç dalga $\times$ 200 ms $\approx$ 600 ms sürer. Sınır olmasaydı yaklaşık 200 ms sürerdi.

Testi: `resilience/CoverServiceTest`

★ İpucu

`application.yaml` içinde `org.springframework.resilience: debug` satırını açarsanız, çağırılarının nasıl bekletilip bırakıldığını logda görürsünüz.

3.8 Zaman Aşırımları

Amaç: Cevap vermeyen bir servisin uygulamanızı kitlemesini önlemek.

Varsayılanlar `spring.http.clients.*` altındadır. Bir grup için `spring.http.serviceclient.<grup>.*` ile ayrı değerler verilebilir (bölüm 3.4'teki YAML). Test, okuma zaman aşımını 300 ms yapar ve 2 saniye geciken bir cevabı bekler. Çağrı, 2 saniye beklemek yerine kısa sürede `ResourceAccessException` ile biter:

```

@TestPropertySource(properties = "spring.http.clients.read-timeout=300ms")
class TimeoutTest extends WireMockCatalogTest {

    @Autowired
    CatalogRestClient client;

    @Test

```

```

void aSlowResponseTimesOut() {
    cata-
    ↪ log.stubFor(get("/catalog/books/9780134685991").willReturn(okJson(EFFECTIVE_JAVA).withFixedDelay(2

    long start = System.nanoTime();
    assertThatThrownBy(() ->
        ↪ client.find("9780134685991")).isInstanceOf(ResourceAccessException.class);
    org.assertj.core.api.Assertions.assertThat((System.nanoTime() - start) /
    ↪ 1_000_000).isLessThan(1_500);
}
}

```

!! Dikkat

Zaman aşımı olmayan bir HTTP çağrısı, uzak servis takıldığında thread'inizi (ve virtual thread'lerle bile bağlantınızı ve belleğinizi) süresiz tutar. Her istemci için bilinçli bir zaman aşımı ayarlayın.

4. Sık Yapılan Hatalar ve En İyi Pratikler

! Uyarı

Spring Test, test bağlamlarını önbelleğe alır. WireMock'u JUnit eklentisiyle **her test sınıfı için** başlatırsanız, her sınıf yeni bir rastgele port alır. Önbellekteki bağlam ise eski portu kullanmaya devam eder ve "Connection refused" hatası alırsınız. Bu modülün testleri tek bir WireMock sunucusunu JVM başına bir kez başlatır (WireMockCatalogTest).

- **Yapın:** Boot'un verdiği `RestClient.Builder`'ı kullanın, `RestClient.create()` ile sıfırdan oluşturmayın. Ayarları ve customizer'ları kaybedersiniz.
- **Yapmayın:** HTTP hata sınıflarını (`HttpClientErrorException`) iş katmanına sızdırmayın. `onStatus` ile domain exception'larına çevirin.
- **Yapın:** Yalnızca geçici hataları, sınırlı sayıda ve artan beklemeyle (`multiplier`, `jitter`) tekrar deneyin.
- **Yapmayın:** Aynı çağrıyı hem istemcide hem de çağırılan serviste tekrar denemeyin. Denemeler çarpılır.
- **Yapın:** Harici servisleri testlerde WireMock gibi bir sahte sunucuyla taklit edin ve gelen istekleri (`verify`) doğrulayın.

5. Özet

- `RestClient` varsayılan HTTP istemcisidir. `onStatus` HTTP hatalarını domain exception'larına çevirir.

- `RestClientCustomizer`, tüm istemcilere tek yerden ayar ekler.
- HTTP interface'ler ve `@ImportHttpServices` ile istemci kodu yazmadan servis çağrılır. Ayarlar `spring.http.serviceclient.<grup>` altındadır.
- Spring Framework 7'de `@Retryable` ve `@ConcurrencyLimit` yerleşiktir ve `@EnableResilientMethods` ile açılır.
- Zaman aşırımları `spring.http.clients.*` ile ayarlanır.
- `WireMock`, dış servisleri güvenilir biçimde taklit eder.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Framework — REST Clients](#)
- [Spring Framework — Resilience Features](#)
- [Spring Boot — Calling REST Services](#)
- [WireMock Documentation](#)